CSA0609-DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT – 1

Name: Hemanth P
Register Number: 192524297

`

Task 1 – Design the Algorithm

Aim

To design a recursive Binary Search algorithm to locate a student's registration number in a large database efficiently.

Basic Idea

Binary Search finds a target in a sorted list by comparing the target with the middle element. If the target is smaller, the search continues in the left half. If it is larger, the search continues in the right half. Each step reduces the search interval to approximately half its previous size.

Design Strategy

- Algorithm technique: Divide and conquer.
- Implementation: Recursive.
- Input: Sorted registration numbers and a target registration number.
- Output: Index of the target if found; otherwise, −1.
- Preprocessing: Registration numbers must be sorted before searching if they are initially unsorted.

Pseudocode

```
BINARY-SEARCH(A, target, low, high)
  if low > high
    return -1
  mid = floor((low + high) / 2)
  if A[mid] = target
    return mid
  if target < A[mid]
    return BINARY-SEARCH(A, target, low, mid - 1)
  else
    return BINARY-SEARCH(A, target, mid + 1, high)
```

Step-by-Step Example

Given the sorted registration numbers: A = [10, 20, 30, 40, 50, 60, 70]. Target registration number: 60.

Step 1: low = 0, high = 6. The middle index is floor((0 + 6) / 2) = 3, and A[3] = 40. Since 60 > 40, search the right half.

Step 2: Update low = mid + 1 = 4 and high = 6. The new middle index is floor((4 + 6) / 2) = 5, and A[5] = 60.

Step 3: The target matches the middle element. Registration number 60 is found at index 5 using two comparisons.

Justification

Binary Search is suitable for a large student database because it eliminates approximately half of the remaining records after each unsuccessful comparison. It requires sorted data, but once the records are sorted, repeated searches can be performed efficiently.

Task 2 – Identify the Basic Operation:

- The basic operation is the comparison between the target registration number and the middle element. It occurs in the equality test and the comparison that determines whether the algorithm should search the left or right half.

Explanation

- The algorithm calculates the middle index.
- It compares the middle element with the target.
- If they match, the search stops.
- If they do not match, the algorithm selects one half of the remaining interval.
- The process repeats until the target is found or the interval becomes empty.

The number of comparisons increases logarithmically because the search interval is halved after each step. For a sorted dataset containing 1,000,000 records, the search requires about 20 halving steps along a search path.

Task 3 – Derive the Time-Complexity Function Mathematically:

Let n be the number of registration numbers in the sorted array.

Step 1: Identify the Size of the Problem

Initially, the algorithm searches n elements. After one comparison, the remaining interval contains approximately n/2 elements. After two comparisons, it contains $n/2^2 = n/4$ elements. After three comparisons, it contains $n/2^3 = n/8$ elements.

Step 2: Express the Remaining Interval

After k comparisons, the remaining interval is approximately $n/2^k$.

Step 3: Identify the Stopping Condition

The search reaches one candidate element when $n/2^k = 1$. Therefore, $n = 2^k$.

Step 4: Take Logarithms

Taking logarithms to base 2 gives $\log_2 n = k$. Hence, $k = \log_2 n$.

Step 5: Derive the Complexity

The algorithm performs a constant amount of work per comparison. The worst-case running time can be expressed as $T(n) = c \log_2 n + d$, where c and d are constants. Ignoring constant factors and lower-order terms, $T(n) = \Theta(\log n)$.

Step 6: Include Preprocessing Cost

If the registration numbers are already sorted, the search takes $\Theta(\log n)$. If the records are initially unsorted, an efficient comparison sort generally takes $\Theta(n \log n)$. For one search after sorting, $T_{total}(n) = \Theta(n \log n) + \Theta(\log n) = \Theta(n \log n)$. If the sorted records are reused for many searches, the sorting cost is paid once rather than for every request.

Task 4 – Analyse Best, Average and Worst-Case Behaviour:

Binary Search has different performance depending on the target's position and whether it exists in the array.

4.1 Best Case

The best case occurs when the target is exactly at the middle of the array during the first comparison. The algorithm performs one comparison: $C_{best}(n) = 1$. Therefore, $T_{best}(n) = \Theta(1)$.

4.2 Average Case

The average case occurs when the target is found after several divisions of the search interval. Since approximately half the remaining elements are eliminated at each step, the average running time is $\Theta(\log n)$, assuming targets are reasonably distributed across the sorted data.

4.3 Worst Case

The worst case occurs when the algorithm follows the deepest search path or when an unsuccessful search continues until the interval becomes empty. The number of comparisons is bounded by approximately $\text{floor}(\log_2 n) + 1$. Therefore, $T_{worst}(n) = \Theta(\log n)$.

4.4 Comparison Table

Case	Condition	Time complexity
Best	Target is the first middle element checked	$\Theta(1)$
Average	Target is found after several divisions	$\Theta(\log n)$
Worst	Deepest path or unsuccessful search	$\Theta(\log n)$

Conclusion: Binary Search is efficient because even its worst-case search time grows logarithmically. However, the input must be sorted before searching.

Task 5 – Perform Asymptotic Analysis:

Asymptotic analysis describes the growth of an algorithm's running time as the input size becomes large.

5.1 Big-O Notation

Big-O describes an asymptotic upper bound. For Binary Search, $T(n) = O(\log n)$.

5.2 Big-Omega Notation

Big-Omega describes an asymptotic lower bound. The worst-case number of comparisons is $\Omega(\log n)$. This lower bound does not describe every individual search, because the target may be found immediately.

5.3 Big-Theta Notation

Big-Theta describes a tight asymptotic bound. The worst-case number of comparisons is $\Theta(\log n)$, meaning it grows proportionally to log n within constant factors.

5.4 Growth for Different Dataset Sizes

Number of records	**$\log_2(n)$**	**Approx. worst-case comparisons**
1,000	9.97	10–11
10,000	13.29	14
100,000	16.61	17–18
1,000,000	19.93	20–21

The exact comparison count depends on the array size, implementation and target. Increasing the database from 1,000 to 1,000,000 records multiplies its size by 1,000, but adds only about 10 comparison levels. This makes Binary Search suitable for large sorted datasets.

Task 6 – Formulate the Recurrence Relation:

A recurrence relation represents the running time of a recursive algorithm in terms of a smaller problem.

Derivation

- Original input size: n elements in the current search interval.
- Recursive subproblem: approximately n/2 elements.
- Number of recursive calls: at most one per step.
- Work outside the recursive call: constant time to check boundaries, calculate the middle index, compare values and choose a half.
- Base case: the target is found or the interval becomes empty; this takes constant time.

For $n > 1$, $T(n) = T(n/2) + c$, where c is a constant. The base case is $T(1) = c_0$, where c_0 is a constant.

Therefore: $T(n) = c_0$ for $n = 1$; and $T(n) = T(n/2) + c$ for $n > 1$.

Conclusion: The recurrence follows directly from Binary Search's behaviour: one smaller subproblem of approximately half the size and constant work outside the recursive call.

Task 7 – Solve the Recurrence Relation:

We solve $T(n) = T(n/2) + c$ using the iteration method, which is appropriate because each recursive call reduces the problem size by half and adds constant work.

Step 1: Expand the Recurrence
$T(n) = T(n/2) + c$.

Since $T(n/2) = T(n/4) + c$, substituting gives $T(n) = T(n/4) + 2c$.

Step 2: Expand Again
$T(n/4) = T(n/8) + c$. Therefore, $T(n) = T(n/8) + 3c$.

Step 3: Identify the Pattern
After k expansions, $T(n) = T(n/2^k) + kc$.

`

Step 4: Find the Stopping Condition
The recursion stops when $n/2^k = 1$. Thus $n = 2^k$ and $k = \log_2 n$.

Step 5: Substitute
$T(n) = T(1) + c \log_2 n$. Since T(1) and c are constants, $T(n) = \Theta(\log n)$.

Interpretation: Binary Search has logarithmic worst-case time complexity. The number of recursive calls increases slowly even when the dataset grows from thousands to millions of records.

Task 8 – Implement and Test the Algorithm Using Python:

Aim
To implement recursive Binary Search in Python and measure its performance for different dataset sizes.

Python Program:

```
import time
def binary_search(data, target, low, high, counter):
    if low > high:
        return -1
    mid = (low + high) // 2
    counter[0] += 1
    if data[mid] == target:
        return mid
    if target < data[mid]:
        return binary_search(data, target, low, mid - 1, counter)
    return binary_search(data, target, mid + 1, high, counter)

sizes = [1000, 10000, 100000, 1000000]
for n in sizes:
    data = list(range(100000000, 100000000 + n))
    target = data[-1]
    counter = [0]
    start = time.perf_counter()
    result = binary_search(data, target, 0, len(data) - 1, counter)
    elapsed = time.perf_counter() - start
```

```
print("Records:", n)
print("Target:", target)
print("Index:", result)
print("Comparisons:", counter[0])
print("Execution time:", elapsed, "seconds")
print("-" * 30)
```

Procedure

- Create sorted registration numbers for each dataset.
- Select the last registration number as the search target.
- Initialise the comparison counter to zero.
- Execute the recursive Binary Search function.
- Record the index returned by the algorithm.
- Measure execution time using time.perf_counter().
- Repeat the experiment for all four dataset sizes.
- Compare the results with the theoretical complexity.

Expected Result:

The algorithm should locate the target successfully for all four datasets. The comparison count should increase logarithmically as the dataset size increases. Execution times must be measured by running the program on the actual computer; they may vary with hardware and system load.

Task 9 – Record Experimental Performance:

The experiment evaluates Binary Search using datasets of 1,000, 10,000, 100,000 and 1,000,000 records.

9.1 Observation Table

Run the Python program from Task 8 and enter the actual comparison counts and execution times. No execution-time measurements are invented in this report.

Number of records	Target position	Comparisons observed	Execution time
1,000	Last element	Record from program	Record from program
10,000	Last element	Record from program	Record from program
100,000	Last element	Record from program	Record from program
1,000,000	Last element	Record from program	Record from program

9.2 Analysis

- Dataset size increases by a factor of 10 between consecutive experiments.
- The search interval is halved after each unsuccessful comparison.
- The number of comparisons grows much more slowly than the number of records.
- Execution time may vary because of processor speed, memory access and system load.
- The comparison counter provides a more direct measure of algorithmic work than a single execution-time measurement.

9.3 Result

Binary Search should find the target registration number in every test. The observations should demonstrate logarithmic growth in comparison count, consistent with $T(n) = \Theta(\log n)$.

Task 10 – Compare Theoretical and Experimental Results

10.1 Theoretical Analysis

The theoretical worst-case time complexity of Binary Search is $T(n) = \Theta(\log n)$. Each recursive call reduces the search interval to approximately half its previous size.

10.2 Experimental Analysis

The program measures the number of comparisons and execution time for increasing dataset sizes. The expected comparison counts are logarithmic in scale. For one million elements, the maximum search depth is approximately 20 halving steps, with the exact count depending on the implementation and target.

10.3 Comparison Table

Factor	**Theoretical result**	**Experimental observation**
Search strategy	Repeatedly halves the interval	Middle index recalculated per call
Comparison growth	Logarithmic	Should increase slowly
Execution time	Depends on implementation constants	Depends on computer and system load
Data requirement	Sorted input	Program uses sorted input
Correctness	Returns index or −1	Verify returned index and value

10.4 Discussion

The theoretical analysis explains how the number of operations grows with input size. Experimental results show the algorithm's actual behaviour on a particular computer. A difference in measured execution time does not necessarily mean that the theoretical analysis is incorrect. Hardware, interpreter overhead and system activity affect measured time.

Conclusion: The theoretical and experimental results should agree on the overall trend: Binary Search needs only a logarithmic number of comparisons on sorted data.

Task 11 – Recommend and Justify the Final Design

11.1 Recommended Algorithm

Recursive Binary Search is recommended for retrieving student records when registration numbers are maintained in sorted order.

11.2 Reasons for Selection

1. Efficient searching: Binary Search reduces the search interval by half at every step, resulting in logarithmic worst-case search time.
2. Suitable for large datasets: It remains efficient when the database grows from 1,000 to 1,000,000 records.
3. Low additional memory requirement: The recursive implementation requires a call stack of depth $\Theta(\log n)$.
4. Repeated searches: When sorted data is maintained and reused, repeated searches do not require sorting the entire dataset again.
5. Clear mathematical analysis: The recurrence $T(n) = T(n/2) + c$ solves to $T(n) = \Theta(\log n)$.

11.3 Limitations

- Registration numbers must be sorted.
- Sorting initially unsorted data adds preprocessing cost.
- Inserting new records into a sorted array may require shifting elements.
- The system must ensure registration numbers are unique and sorted order is maintained.

Final Decision

Use recursive Binary Search for exact registration-number searches. Maintain records in sorted order and include the cost of sorting or updating that order in the system's overall efficiency analysis.

Task 12 – Day-4 Design Challenge

Scenario: The university must process 50,000 search requests per hour using a database containing 1,000,000 student records. New student records are added once per day.

A. Impact Analysis

The average search-request rate is 50,000 / 3,600 = 13.89 requests per second, or approximately 14 requests per second on average. The database contains one million records, so searching sequentially through every record would be costly. Binary Search is suitable when registration numbers are sorted.

B. Evaluate the Existing Design

For one million records, $\log_2(1{,}000{,}000) \approx 19.93$, indicating approximately 20 halving steps to reach one candidate element.

For 50,000 requests, the rough estimate is 50,000 × 20 = 1,000,000 comparison steps. The exact total depends on target positions and the number of comparisons per search.

C. Adaptation of the Design

Decision: Retain Binary Search and improve how sorted records are maintained.

- Load student registration numbers from the university database.
- Sort the registration numbers before the first search if necessary.
- Use recursive Binary Search for each incoming request.
- Maintain sorted order when new records are added daily.
- Validate each search result against the stored registration number.
- Measure response time under realistic workloads, including peak traffic.

D. Preprocessing Cost

If records are initially unsorted, an efficient comparison sort generally requires $T_{sort}(n) = \Theta(n \log n)$. Each subsequent Binary Search requires $T_{search}(n) = \Theta(\log n)$. For q searches after sorting, $T(n, q) = \Theta(n \log n) + q\Theta(\log n) = \Theta(n \log n + q \log n)$. This assumes sorted data can be reused and does not need to be sorted again for every request.

E. Defence of the Final Design

- Mathematical evidence: Binary Search has logarithmic worst-case search complexity.
- Dataset size: One million sorted records are suitable for binary searching.

- Search frequency: Sorted order can be reused across 50,000 requests per hour.
- Update frequency: Daily additions allow the university to plan for maintaining sorted order.
- Preprocessing: Sorting and update costs must be included in the complete system analysis.
- Practical validation: Actual response times should be measured on target hardware under representative load.

Final Conclusion:

Binary Search is an appropriate choice for the stated workload, provided registration numbers remain sorted. The university should retain the algorithm, maintain sorted data efficiently, and measure real response times to confirm that the system meets operational requirements.

Overall Conclusion

This assignment designed, analysed and described testing for recursive Binary Search in a university student-record system. The mathematical derivation, recurrence relation and asymptotic analysis show that its worst-case search time is $\Theta(\log n)$.

The Python experiment provides a way to validate the theoretical analysis using different dataset sizes. For the design challenge involving one million records and 50,000 searches per hour, Binary Search remains suitable when records are maintained in sorted order and preprocessing costs are properly considered.